Control Structures: Loops and Conditionals

In this lecture, we will explore control structures in Python, focusing on loops and conditional statements. Control structures are essential for creating dynamic and responsive code, allowing us to implement repetition and selection in our programs. By the end of this lecture, you will understand how to use 'for' and 'while' loops, as well as 'if', 'elif', and 'else' statements.

Introduction to Control Structures

Control structures are constructs that dictate the flow of control in a program. They allow us to execute certain blocks of code based on specific conditions or to repeat blocks of code multiple times. In Python, the primary control structures we will cover are loops and conditionals.

Loops

Loops are used to execute a block of code repeatedly. Python provides two types of loops: 'for' loops and 'while' loops.

For Loop

The 'for' loop is used to iterate over a sequence (like a list, tuple, or string) or a range of numbers. The syntax for a 'for' loop is as follows:

```
for variableName in groupOfValues:
statements
```

Example:

```
for x in range(1, 6):
print(x, "squared is", x * x)
```

Output:

```
1 squared is 1
2 squared is 4
3 squared is 9
4 squared is 16
5 squared is 25
```

The `range()` function generates a sequence of numbers. In the example above, it generates numbers from 1 to 5.

While Loop

The 'while' loop continues to execute a block of code as long as a specified condition is true. The syntax is:

```
while condition:
statements
```

Example:

```
number = 1
while number < 200:
print(number)
number = number * 2
```

Output:

```
1
2
4
```

```
8
16
32
64
128
```

Conditionals

Conditional statements allow us to execute certain blocks of code based on whether a condition is true or false. The primary conditional statements in Python are 'if', 'elif', and 'else'.

If Statement

The 'if' statement executes a block of code if a specified condition is true. The syntax is:

```
if condition:
statements
```

Example:

```
gpa = 3.4
if gpa > 2.0:
print("Your application is accepted.")
```

If/Else Statement

The 'if/else' statement allows us to execute one block of code if the condition is true and another block if it is false. The syntax is:

```
if condition:
statements
else:
statements
```

Example:

```
gpa = 1.4
if gpa > 2.0:
print("Welcome to Mars University!")
else:
print("Your application is denied.")
```

Elif Statement

The 'elif' statement allows us to check multiple conditions. The syntax is:

```
if condition:
statements
elif condition:
statements
else:
statements
```

Example:

```
x = 30
if x <= 15:
y = x + 15
elif x <= 30:
y = x + 30
else:
y = x
```

```
print('y =', y)
```

Combining Loops and Conditionals

Loops and conditionals can be combined to create more complex logic in your programs. For example, you can use a loop to iterate through a list of numbers and use a conditional statement to check if each number is even or odd.

Example:

```
for i in range(1, 11):
if i % 2 == 0:
print(i, "is even")
else:
print(i, "is odd")
```

Conclusion

In this lecture, we covered the fundamental control structures in Python: loops and conditionals. We learned how to use 'for' and 'while' loops to implement repetition and how to use 'if', 'elif', and 'else' statements to implement selection. Mastering these concepts is crucial for writing dynamic and responsive Python programs. In the next lecture, we will explore functions and how they can help us organize our code more effectively.

Quiz Questions

Q1: What are the two types of loops provided by Python?

- For and Until loops
- For and While loops
- Do While and For loops
- If and Else loops

Correct: For and While loops

Q2: Which statement is used to execute a block of code if a specified condition is true?

- elif
- else
- if
- while

Correct: if

Q3: What does the 'range()' function do in a for loop?

- Generates a random number
- Creates a list of strings
- Generates a sequence of numbers
- Counts the number of iterations

Correct: Generates a sequence of numbers